

Advanced Relational Database Architecture: A Comprehensive Analysis of Normalization and PL/SQL Procedural Logic

1. Introduction to Database Systems Engineering

The architecture of modern information systems relies fundamentally on the efficient storage, retrieval, and manipulation of data. This report provides an exhaustive technical analysis of Unit III of the Database Management Systems curriculum, focusing on two distinct but complementary pillars of database engineering:

Normalization Theory and Procedural Language/Structured Query Language (PL/SQL) Programming.

The first pillar, Normalization, represents the mathematical and theoretical foundation of the relational model. It is the rigorous process of structuring data to minimize redundancy and maintain data integrity.¹ Derived from the seminal works of Codd, Elmasri, Navathe, and Silberschatz, normalization utilizes the concept of functional dependencies to decompose relations into their most atomic, logical forms, thereby preventing insertion, update, and deletion anomalies.³

The second pillar, PL/SQL, represents the practical, computational extension of the database. While standard SQL is declarative—focusing on *what* data to retrieve—PL/SQL provides the procedural logic—the *how*—allowing for complex control structures, iterative processing, and event-driven programming via triggers and cursors.⁵ This report synthesizes these domains to provide a holistic view of robust database application design.

2. Relational Database Design and Normalization Theory

Relational database design is not merely a method of storage; it is a discipline of semantic preservation. The primary objective is to create a database schema where every relation (table) represents a single, cohesive entity or relationship type, ensuring that facts are recorded exactly once.³ This reduction of redundancy is critical not for saving storage space, which is cheap, but for preventing logical inconsistencies known as anomalies.

2.1 The Theory of Redundancy and Anomalies

In an unnormalized or poorly designed database, information is often conflated. A single relation might attempt to store attributes of multiple distinct entities, such as combining Student, Course, and Instructor data into a single "flat" table. This conflation leads to data redundancy, where the same fact (e.g., an instructor's office address) is repeated for every course they teach.¹

This redundancy breeds anomalies, which are structural flaws that compromise data integrity during Data Manipulation Language (DML) operations:

- **Update Anomalies:** These occur when a single logical fact is stored in multiple rows. If an instructor changes their office, the system must update

this information in every single row representing a course taught by that instructor. Failure to update even one row results in a database that asserts contradictory facts—the instructor is simultaneously in two different offices.¹

- **Insertion Anomalies:** These occur when the structure of the table prevents the addition of certain data due to primary key constraints. In a combined Student-Course table where the primary key is the composite {Student_ID, Course_ID}, it becomes impossible to add a new student to the system until they formally register for a course. The system cannot record the student's existence because part of the primary key (Course_ID) would be null, violating Entity Integrity.¹
- **Deletion Anomalies:** This is the inverse of the insertion anomaly. If a student drops their only course, deleting the enrollment record may inadvertently result in the permanent loss of the student's personal information (Name, Address) if that data is not stored in a separate, independent relation.¹

Normalization resolves these issues by decomposing large relations into smaller, well-structured relations that adhere to specific "Normal Forms," ensuring that each table governs exactly one theme.

2.2 Functional Dependencies (FDs)

The engine driving the normalization process is the concept of Functional Dependency (FD). An FD is a constraint that describes the relationship between attributes within a relation. It asserts that the value of one set of attributes (the determinant) uniquely determines the value of another set of attributes (the dependent).⁸

Formal Definition:

Let R be a relation schema, and let X and Y be subsets of the attributes of R . The functional dependency $X \rightarrow Y$ holds on R if and only if for every legal relation instance $r(R)$, whenever two tuples t_1 and t_2 agree on the value of X , they must also agree on the value of Y .

$$\forall t_1, t_2 \in r(R): t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

.4

This mathematical definition implies that X acts as a key for Y . For example, in a university database, Student_ID functionally determines Student_Name ($\text{Student_ID} \rightarrow \text{Student_Name}$). It is impossible for the same Student ID to be associated with two different names in the same instance.⁹

2.2.1 Types of Functional Dependencies

Understanding the nuance of dependencies is crucial for identifying normal form violations:

1. **Full Functional Dependency:** An attribute is fully functionally dependent on a set of attributes X if it is dependent on X and not on any proper subset of X . This is the goal for all non-key attributes in a relation.⁹
2. **Partial Dependency:** An attribute is partially dependent if it depends on only a part of a composite primary key. If a key is {Student_ID, Course_ID} and we store Instructor_Name, and Instructor_Name is determined solely by Course_ID, this is a partial dependency.⁹

3. **Transitive Dependency:** This occurs when dependencies cascade. If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ is a transitive dependency. Here, Z depends on X only through the intermediary Y .⁹
4. **Trivial Dependency:** A dependency $X \rightarrow Y$ is trivial if Y is a subset of X (e.g., $\{Student_ID, Name\} \rightarrow Student_ID$). These are always true and do not provide constraint information.⁴

2.2.2 Inference Rules: Armstrong's Axioms

To manipulate and reason about dependencies, database designers use a set of inference rules known as Armstrong's Axioms. These rules are proven to be sound (they generate only correct dependencies) and complete (they can generate all valid dependencies derived from a set).¹

Axiom Name	Rule	Explanation
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$	Any set of attributes determines itself or its subsets. This forms the basis of trivial dependencies.
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	Adding the same attributes (Z) to both sides of a dependency preserves the relationship.
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	Dependencies propagate through intermediaries.

From these primary axioms, secondary rules are derived to simplify analysis:

- **Union (Additivity):** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$. This allows us to combine dependencies with the same determinant.¹
- **Decomposition (Projectivity):** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$. This allows us to split composite dependents.¹
- **Pseudotransitivity:** If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$. This allows for the substitution of determinants in complex chains.¹

2.2.3 Closure of Attributes and Keys

The concept of Closure, denoted as X^+ , is essential for finding Candidate Keys. The closure of a set of attributes X under a set of dependencies F is the set of all attributes that are functionally determined by X .

Algorithmically, one computes closure by repeatedly applying Armstrong's axioms (specifically transitivity and union) until no new attributes can be added to the set. If the closure X^+ includes all attributes in the relation R , then X is a Superkey of R . If no proper subset of X is a superkey, then X is a Candidate Key.¹

2.3 The Hierarchy of Normal Forms

Normalization is a progressive filtering process. A relation is evaluated against specific criteria (Normal Forms). If it fails, it is decomposed into two or more smaller relations that satisfy the criteria.

2.3.1 First Normal Form (1NF): Atomicity

Definition: A relation is in 1NF if and only if the domain of each attribute contains only atomic (indivisible) values, and the value of each attribute in a tuple is a single value from that domain.¹

Implications:

1NF explicitly forbids:

- **Repeating Groups:** Multiple fields for the same type of data (e.g., Phone1, Phone2, Phone3).
- **Multi-valued Attributes:** Storing multiple values in a single cell (e.g., "Java, C++, Python" in a Skills column).
- **Composite Attributes:** Fields that can be meaningfully divided (e.g., storing "New York, NY" in a single Location field instead of separating City and State).

Violation and Resolution:

Consider an Employee table where the Skills attribute contains comma-separated values. This design complicates queries (e.g., "Find all employees who know Java" requires pattern matching rather than direct equality checks).

To normalize to 1NF, the designer must decompose the table. The standard solution is to create a separate relation, say Employee_Skills, with a composite primary key {Employee_ID, Skill}. This transformation ensures that each skill is stored in its own row, restoring atomicity and simplifying data retrieval.²

2.3.2 Second Normal Form (2NF): Full Functional Dependency

Definition: A relation is in 2NF if it is in 1NF and every *non-prime* attribute (an attribute that is not part of any candidate key) is fully functionally dependent on the primary key.¹

The Problem Addressed:

2NF specifically targets redundancy arising from Composite Primary Keys. If a relation has a single-attribute primary key, it is automatically in 2NF (barring non-prime dependencies, which are covered in 3NF). Violations occur when a non-prime attribute depends on only part of a composite key (Partial Dependency).

Example Scenario:

Consider a relation Grades(Student_ID, Course_ID, Professor_Name) where the primary key is {Student_ID, Course_ID}.

- The dependency {Student_ID, Course_ID} → Grade is a full dependency; the grade relates to the specific student in the specific course.
- However, Course_ID → Professor_Name also exists (assuming one professor per course). Here, Professor_Name depends only on Course_ID, a subset of the primary key.

Resolution:

This partial dependency means Professor_Name is repeated for every student taking the course. To normalize to 2NF, we decompose the relation:

1. Enrollment(Student_ID, Course_ID, Grade)
2. Course_Faculty(Course_ID, Professor_Name)

This separation ensures that if the professor for a course changes, only one row in Course_Faculty needs to be updated, eliminating the update anomaly.⁴

2.3.3 Third Normal Form (3NF): Transitive Dependency

Definition: A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key.

Formally, for every functional dependency $X \rightarrow A$ in the relation, one of the following conditions must hold:

1. $X \rightarrow A$ is trivial (i.e., $A \in X$).
2. X is a superkey of the relation.
3. A is a prime attribute (part of a candidate key).¹

The Problem Addressed:

3NF targets dependencies between non-prime attributes. If a primary key determines Attribute A, and Attribute A determines Attribute B, then Attribute B is transitively dependent on the key. This structure creates redundancy because the relationship between A and B is repeated for every occurrence of the primary key.

Example Scenario:

Consider Employee(Emp_ID, Zip_Code, City, State).

- Emp_ID is the primary key.
- Emp_ID $\rightarrow$ Zip_Code holds.
- However, Zip_Code $\rightarrow$ {City, State} also holds (in the US postal system).
- Therefore, Emp_ID $\rightarrow$ {City, State} is a transitive dependency via Zip_Code.

Resolution:

Decompose into:

1. Employee(Emp_ID, Zip_Code)
2. Location(Zip_Code, City, State)

This ensures that "City" and "State" are stored only once per "Zip_Code", rather than once per employee.⁷

2.3.4 Boyce-Codd Normal Form (BCNF)

Definition: BCNF is a stricter generalization of 3NF. A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow A$, X is a superkey.¹

BCNF vs. 3NF:

The critical difference lies in the third condition of the 3NF definition: "A is a prime attribute." 3NF allows a dependency $X \rightarrow A$ where X is not a key, provided that A is part of a key. BCNF removes this loophole. BCNF requires that the determinant (X) must be a superkey, regardless of the status of A .²

Violation Example:

Consider `Student_Project(Student, Project, Mentor)`.

Assumptions:

1. A student can work on many projects, but only one mentor per project.
2. A mentor manages only one project.
3. A student works with a specific mentor for a specific project.

Key: {Student, Project}.

If there is a dependency `Mentor -> Project` (because a mentor leads only one project), but `Mentor` is not a superkey (knowing the mentor doesn't identify the student), this violates BCNF. It would be allowed in 3NF because `Project` is a prime attribute (part of the candidate key).

Decomposition and Trade-offs:

Decomposing to BCNF guarantees the removal of all redundancy based on functional dependencies. However, it may not always be Dependency Preserving. In the example above, splitting the table might make it difficult to enforce the constraint that a student stays with the same mentor for a project without joining tables. Thus, 3NF is sometimes preferred in practice when dependency preservation is prioritized over strict non-redundancy.³

2.4 Advanced Normalization: Join and Multi-Valued Dependencies

Beyond BCNF, we encounter dependencies that involve sets of values rather than single atomic values.

2.4.1 Multi-Valued Dependencies (MVDs) and Fourth Normal Form (4NF)

Multi-Valued Dependency Definition:

An MVD, denoted $X \twoheadrightarrow Y$, exists in a relation $R(X, Y, Z)$ if, for a given value of X , there is a set of values for Y that is associated with X , and this set of Y values is independent of the values of Z .¹

Fourth Normal Form (4NF):

A relation is in 4NF if it is in BCNF and contains no non-trivial multi-valued dependencies.¹¹

Violation Example:

Consider Course_Info(Course, Textbook, Instructor).

- A Course uses a set of Textbooks (e.g., Database Systems, SQL Guide).
- A Course is taught by a set of Instructors (e.g., Prof. A, Prof. B).
- The textbooks used are the same regardless of who teaches the course (Independence).

If we store this in one table, we must create a Cartesian product of every instructor paired with every textbook for that course. If a course has 3 books and 2 instructors, we need 6 rows. If we add a new book, we must add 2 new rows (one for each instructor).

This redundancy is addressed by 4NF decomposition into two tables:

1. Course_Books(Course, Textbook)
2. Course_Instructors(Course, Instructor)

This eliminates the Cartesian product and allows independent updates.¹⁴

2.4.2 Join Dependencies and Fifth Normal Form (5NF)

Join Dependency Definition:

A Join Dependency (JD), denoted $*(R_1, R_2, \dots, R_n)*$, specifies that a relation R can be perfectly reconstructed by joining its projections $R_1, R_2, \dots, R_n$. An MVD is actually a specific type of JD where $n=2$.¹

Fifth Normal Form (5NF):

Also known as Project-Join Normal Form (PJ/NF). A relation is in 5NF if every non-trivial join dependency is implied by the candidate keys of the relation.¹⁷

5NF deals with **cyclic dependencies** in many-to-many-to-many relationships. Consider three entities: Supplier, Part, and Project. If a rule exists that "If a Supplier supplies a Part, and that Part is used in a Project, and that Supplier works with that Project, then the Supplier *must* supply that Part to that Project," this creates a 3-way dependency. 5NF decomposition breaks this into three binary relations (Supplier-Part, Part-Project, Supplier-Project) to prevent spurious tuples—false records that appear valid when joining but represent relationships that don't actually exist.¹⁸

3. PL/SQL Programming: Architecture and Fundamentals

While normalization ensures the structural integrity of data, standard SQL (Structured Query Language) often falls short when complex application logic is required. SQL is declarative; it effectively states *what* results are desired but not the algorithmic steps to achieve them. To bridge this gap, Oracle developed **PL/SQL (Procedural Language extensions to SQL)**.

PL/SQL integrates the data manipulation power of SQL with the procedural constructs of modern programming languages (variables, loops, conditions, exception handling), allowing for the creation of robust, server-side applications.⁵

3.1 PL/SQL Architecture and Block Structure

PL/SQL is a block-structured language. The basic unit of coding is the **Block**, which groups related declarations and statements. This block structure allows the PL/SQL engine to send an entire block of code to the database server in a single call, drastically reducing network traffic (context switching) compared to sending individual SQL statements one by one.⁶

Structure of a PL/SQL Block:

A standard block consists of three sections:

1. **Declaration Section (Optional):** Defines variables, constants, cursors, and user-defined types. It starts with the keyword `DECLARE`.
2. **Execution Section (Mandatory):** Contains the procedural and SQL statements that perform the logic. It starts with `BEGIN` and ends with `END`.
3. **Exception Handling Section (Optional):** Catches and handles runtime errors. It starts with `EXCEPTION`.

SQL

`DECLARE`

-- Declaration: Define a variable to hold salary

`v_salary NUMBER(8,2);`

`v_id NUMBER := 101;`

`BEGIN`

-- Execution: Retrieve data into the variable

`SELECT salary INTO v_salary`

`FROM employees`

`WHERE employee_id = v_id;`

-- Logic: Conditional check

`IF v_salary < 5000 THEN`

`UPDATE employees SET salary = salary + 500 WHERE employee_id = v_id;`

`END IF;`

`EXCEPTION`

-- Exception Handling: Graceful failure

`WHEN NO_DATA_FOUND THEN`

`DBMS_OUTPUT.PUT_LINE('Employee not found.');`

`END;`

`/`

`.1`

3.2 PL/SQL Language Elements

3.2.1 Identifiers, Variables, and Data Types

PL/SQL is strongly typed. Variables must be declared before use.

- **Scalar Types:** Hold a single value (e.g., `NUMBER`, `VARCHAR2`, `DATE`, `BOOLEAN`). Note that `BOOLEAN` is unique to PL/SQL and cannot be stored in database columns.²¹
- **LOB Types:** Large Objects (`CLOB`, `BLOB`) for storing text or binary data.
- **Composite Types:** Records and Collections (Tables/Arrays).

The Importance of `%TYPE` and `%ROWTYPE`:

To maintain code stability when the underlying database schema changes, PL/SQL offers anchored declarations:

- `variable_name table.column%TYPE`: Declares a variable with the exact same data type as a specific database column. If the column definition changes (e.g., `VARCHAR2(20)` becomes `VARCHAR2(50)`), the variable automatically updates upon recompilation.
- `record_name table%ROWTYPE`: Declares a record variable that matches the structure of an entire row in a table. This is essential for fetching complete rows via cursors.²²

3.2.2 Operators

PL/SQL supports a comprehensive set of operators for data manipulation.

Operator Type	Operators	Description	Usage Example
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code>	Standard math. <code>**</code> is the exponentiation operator.	<code>result := 2 ** 3;</code> <code>-- 8</code>
Relational	<code>=</code> , <code>!=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Comparisons. <code><></code> and <code>!=</code> are strictly equivalent.	<code>IF x <> y</code> <code>THEN...</code>
Logical	<code>AND</code> , <code>OR</code> , <code>NOT</code>	Boolean logic. <code>NOT</code> takes precedence over <code>AND</code> , which takes precedence over <code>OR</code> .	<code>IF (A > B)</code> <code>AND (C < D)...</code>
String	<code> </code> , <code>LIKE</code>	<code> </code> concatenates strings. <code>LIKE</code> performs pattern matching.	<code>name := first </code> <code>' ' last;</code>
Comparison	<code>BETWEEN</code> , <code>IN</code> , <code>IS NULL</code>	Range, set membership, and <code>NULL</code> checking.	<code>IF x</code> <code>BETWEEN 1</code> <code>AND 10...</code>

Precedence Rules:

The order of operations is critical for correct logic evaluation. Exponentiation (`**`) has the highest precedence, followed by multiplication/division, then addition/subtraction. In logical comparisons, comparison operators (`=`, `<`) are evaluated before logical connectors (`NOT`, `AND`, `OR`).²³

4. Control Structures: Logic Flow in PL/SQL

Control structures allow developers to control the execution path of the program, enabling conditional processing and repetitive tasks.

4.1 Conditional Control (IF and CASE)

4.1.1 IF Statements

The IF statement enables branching logic. PL/SQL supports three forms: IF-THEN, IF-THEN-ELSE, and IF-THEN-ELSIF.

Syntax Note: The keyword is spelled ELSIF, not ELSEIF.

```
SQL
IF v_score >= 90 THEN
    v_grade := 'A';
ELSIF v_score >= 80 THEN
    v_grade := 'B';
ELSE
    v_grade := 'C';
END IF;
```

Handling NULLs: A common pitfall in conditional logic is the handling of NULL values. In PL/SQL, NULL is "unknown." Comparisons like IF variable = NULL will never evaluate to TRUE; one must use IF variable IS NULL.²¹

4.1.2 CASE Statements

The CASE statement is often more readable than complex IF ladders.

- **Simple CASE:** Evaluates a single expression against a list of values.
- **Searched CASE:** Evaluates a list of independent Boolean conditions.

```
SQL
-- Simple CASE Example
CASE v_dept
    WHEN 10 THEN v_dept_name := 'Accounting';
    WHEN 20 THEN v_dept_name := 'Research';
    ELSE v_dept_name := 'Operations';
END CASE;
```

.²⁷

4.2 Iterative Control (Loops)

Loops allow a block of code to be executed repeatedly.

1. **Basic LOOP:** An infinite loop structure that relies on an explicit EXIT statement.

```
SQL
LOOP
    v_counter := v_counter + 1;
    EXIT WHEN v_counter > 10;
END LOOP;
```

2. **WHILE LOOP:** Evaluates a condition *before* executing the loop body. If the condition is false initially, the loop never runs.

```
SQL
WHILE v_salary < 10000 LOOP
    v_salary := v_salary * 1.10;
END LOOP;
```

3. **FOR LOOP:** Iterates over a defined range of integers. The loop counter is implicitly declared (no need to define it in the DECLARE section) and is read-only within the loop.

```
SQL
FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE('Iteration: ' |
```

```
| i);
```

```
END LOOP;
```

```
...
```

To iterate backwards, use the REVERSE keyword: FOR i IN REVERSE 1..5.1

5. Cursors: Handling Sets in a Procedural Environment

A fundamental dissonance exists between SQL and procedural languages: SQL is set-oriented (operating on and returning sets of rows), while procedural languages process one record at a time. **Cursors** are the PL/SQL mechanism used to bridge this gap.

A cursor is a pointer to a private SQL area (Context Area) in the system memory that stores information about the processing of a specific SELECT or DML statement.¹

5.1 Implicit Cursors

Oracle automatically manages implicit cursors for all DML statements (INSERT, UPDATE, DELETE) and for SELECT INTO statements that return exactly one row.

Programmers can access status information about these operations using the SQL cursor attributes:

- SQL%FOUND: Returns TRUE if the DML affected at least one row.
- SQL%NOTFOUND: Returns TRUE if no rows were affected.
- SQL%ROWCOUNT: Returns the integer number of rows affected.
- SQL%ISOPEN: Always returns FALSE for implicit cursors because Oracle closes them immediately after execution.³⁰

5.2 Explicit Cursors

For queries that return multiple rows, the programmer must explicitly declare and manage a cursor to process the result set row-by-row.

The Cursor Lifecycle:

1. **DECLARE:** Define the cursor in the declaration section with a specific SELECT statement.

```
CURSOR c_emp IS SELECT emp_id, last_name FROM employees;
```

2. **OPEN:** Execute the query and bind variables. The active set is identified.
3. **FETCH:** Retrieve the current row into variables. Each fetch advances the pointer to the next row.

```
FETCH c_emp INTO v_id, v_name;
```

4. **CLOSE:** Release the memory context.

```
CLOSE c_emp;
```

Cursor Attributes: Explicit cursors have attributes similar to implicit ones, accessed via `cursor_name%ATTRIBUTE` (e.g., `c_emp%NOTFOUND`).

Cursor FOR Loop:

PL/SQL offers a streamlined syntax that automates the Open, Fetch, and Close cycle.

SQL

```
FOR r_emp IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(r_emp.last_name);
END LOOP;
```

This is the preferred method for iterating through cursors due to its simplicity and safety against memory leaks (forgetting to close a cursor).¹

6. Triggers: Event-Driven Logic

Triggers are specialized stored procedures that are implicitly executed ("fired") by the database in response to specific events. They are distinct from standard procedures because they cannot be manually invoked; they are strictly event-driven.³¹

6.1 Trigger Classification and Architecture

Triggers are classified by the event that initiates them and the scope of their execution.

1. Triggering Events:

- **DML Triggers:** Fired by INSERT, UPDATE, or DELETE statements on a table.
- **DDL Triggers:** Fired by schema changes like CREATE, ALTER, or DROP. Useful for auditing schema modifications or preventing unauthorized changes.
- **System/Database Triggers:** Fired by system events such as LOGON, LOGOFF, STARTUP, or SHUTDOWN.³²

2. Timing (DML Triggers):

- **BEFORE:** Executes before the DML operation modifies the data. Ideal for data validation or automatically populating derived fields (e.g., setting a primary key from a sequence).
- **AFTER:** Executes after the DML operation is complete. Ideal for logging, auditing, or cascading changes to other tables.
- **INSTEAD OF:** Used primarily on Views. If a view is complex and not inherently updatable, an INSTEAD OF trigger can intercept the update and manually redirect logic to the underlying base tables.³²

3. Level:

- **Statement-Level (Default):** Fires once for the triggering statement, regardless of how many rows are affected.
- **Row-Level:** Fires once for *each row* affected by the DML statement. This is specified using the FOR EACH ROW clause.³²

6.2 Accessing Row Data: :OLD and :NEW

In Row-Level triggers, PL/SQL provides access to the data state via pseudo-records :OLD and :NEW.

- **:OLD**: References the value of a column *before* the DML operation. (Undefined for INSERT).
- **:NEW**: References the value of a column *after* the DML operation. (Undefined for DELETE).

Example: Auditing Salary Changes

This trigger fires after any update to the salary column, logging the old and new values.

SQL

```
CREATE OR REPLACE TRIGGER trg_audit_salary
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    INSERT INTO salary_audit (emp_id, old_sal, new_sal, change_date)
    VALUES (:OLD.employee_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/
```

.31

Mutating Table Errors:

A common issue in row-level triggers is the "Mutating Table" error (ORA-04091). This occurs when a row-level trigger attempts to query or modify the same table that is currently undergoing change. Because the table state is inconsistent during the update, Oracle blocks this action. This is often resolved using Compound Triggers or temporary tables.²²

7. Modular Programming: Procedures and Functions

Subprograms (Procedures and Functions) are named PL/SQL blocks stored in the database. They promote **Modularity**, **Reusability**, and **Maintainability**.

7.1 Procedures

A Procedure is a subprogram that performs a specific action. It is defined with a header and a body. Procedures are invoked as standalone statements (e.g., EXECUTE procedure_name;) or called from within other blocks.¹

Parameter Modes:

- **IN (Default)**: Passes a value *into* the procedure. The parameter is treated as a constant inside the procedure (Read-only).
- **OUT**: Returns a value *to* the calling environment. The parameter is uninitialized upon entry and must be assigned a value before the procedure ends.
- **IN OUT**: Passes a value in, allows modification, and returns the updated value.²⁰

SQL

```
CREATE OR REPLACE PROCEDURE adjust_salary (
    p_emp_id IN NUMBER,
    p_percent IN NUMBER,
    p_new_sal OUT NUMBER
) IS
BEGIN
```

```

UPDATE employees
SET salary = salary * (1 + p_percent/100)
WHERE employee_id = p_emp_id
RETURNING salary INTO p_new_sal;
END;

```

7.2 Functions

A Function is similar to a procedure but **must** return a single value using the RETURN clause. Functions are designed to compute values and can be used directly within SQL expressions (e.g., inside a SELECT list or WHERE clause), whereas procedures cannot.¹

```

SQL
CREATE OR REPLACE FUNCTION get_annual_sal (p_id NUMBER)
RETURN NUMBER IS
    v_sal NUMBER;
BEGIN
    SELECT salary * 12 INTO v_sal FROM employees WHERE employee_id = p_id;
    RETURN v_sal;
END;

```

.16

8. Conclusion and Synthesis

The effective design and implementation of a database system require a dual mastery of theoretical structure and practical application logic.

Normalization provides the blueprint. By rigorously applying functional dependency rules and decomposing relations from 1NF through 5NF, designers ensure that the database is robust, logically consistent, and free from destructive anomalies. It transforms raw data requirements into a clean, orthogonal schema where every fact is stored in its appropriate place.

PL/SQL breathes life into this schema. It provides the procedural capabilities necessary to manipulate data beyond simple queries. Through the use of block structures, explicit cursors, modular subprograms, and event-driven triggers, developers can enforce complex business rules, automate maintenance tasks, and build high-performance applications directly within the database layer.

Together, these two disciplines form the core competency of the database specialist: the ability to design a system that not only stores data efficiently but also protects its integrity and empowers its usage through sophisticated logic.

Unit IV Database Management Systems: Advanced Transaction Processing, Concurrency, and Distributed Architectures

1. Introduction to Advanced Database Management

The evolution of Database Management Systems (DBMS) has been driven by the increasing demand for high-performance, reliable, and scalable data processing. While introductory database concepts focus on data modeling and basic SQL querying, the operational reality of enterprise systems—such as banking networks, airline reservation systems, and global supply chains—requires a rigorous understanding of how databases handle simultaneous user access and system failures. Unit IV of the syllabus addresses these critical "under-the-hood" mechanisms: Transaction Processing, Concurrency Control, Recovery Systems, Object-Oriented Database Management Systems (OODBMS), Distributed DBMS, and Security.¹

In a single-user environment, data integrity is relatively easy to maintain. However, modern systems are inherently multi-user. If multiple users attempt to read and write the same data item simultaneously, the database can enter an inconsistent state (e.g., the "Lost Update" problem). To prevent this, DBMSs implement sophisticated **Concurrency Control** protocols. Furthermore, hardware is imperfect; disks fail, power is lost, and software crashes. **Recovery Systems** ensure that despite these catastrophes, committed data is never lost (Durability) and partial transactions are cleanly undone (Atomicity).¹

Beyond the relational model, the complexity of modern data—such as multimedia, geospatial data, and engineering designs—has necessitated the rise of **OODBMS**, which integrates object-oriented programming principles directly into the database engine. Similarly, the globalization of business requires **Distributed DBMS**, where data is spread across network nodes to optimize locality and availability. Finally, protecting this concentrated repository of information requires robust **Security** measures grounded in the CIA triad (Confidentiality, Integrity, Availability).³

This report provides a comprehensive, expert-level analysis of these topics, synthesizing theoretical foundations with practical algorithmic implementations.

2. Transaction Processing Concepts

A **Transaction** is the fundamental unit of program execution in a database management system. It represents a logical unit of work—a sequence of operations (reads, writes, calculations) that must be treated as a single, indivisible entity. The classic example is a bank transfer: deducting \$50 from Account A and adding \$50 to Account B. From the user's perspective, this is one action. From the database's perspective, it involves reading A, writing A, reading B, and writing B. If the system fails after writing A but before writing B, money disappears. Transaction processing ensures this does not happen.¹

2.1 The ACID Properties

The integrity of a transaction-processing system relies on four properties, collectively known as the ACID test. Adherence to these properties is the primary contract between the DBMS and the application developer.

2.1.1 Atomicity

Atomicity is the "all-or-nothing" property. It guarantees that a transaction is treated as an indivisible unit of work. Either all of its operations are executed and reflected in the database, or none of them are.

- **Mechanism:** The DBMS maintains a transaction log (typically a Write-Ahead Log). If a transaction fails (aborts) before completion, the recovery manager uses the log to "undo" or "rollback" any write operations that were applied to the disk or memory buffers.
- **Example:** In the fund transfer \$T_1\$, if the system crashes after `Write(A)`, the atomicity property ensures that upon restart, the update to A is reversed, restoring the account to its original balance.¹

2.1.2 Consistency

Consistency ensures that a transaction transforms the database from one valid state to another valid state. A valid state is one that satisfies all defined integrity constraints (e.g., primary keys, foreign keys, domain constraints, and explicit business rules like "Account balance cannot be negative").

- **Responsibility:** Unlike the other properties, consistency is a shared responsibility. The DBMS enforces constraints defined in the schema, but the programmer must ensure the transaction logic is correct (e.g., ensuring the amount deducted from A equals the amount added to B).
- **Implication:** If a transaction violates a constraint (e.g., trying to insert a duplicate primary key), the DBMS will abort the transaction to preserve consistency.²

2.1.3 Isolation

In a multiprogramming environment, multiple transactions execute concurrently. Isolation guarantees that the execution of uncommitted transactions remains invisible to other transactions. Even though they run in parallel, the final result must appear as if they ran serially (one after the other).

- **Mechanism:** This is achieved through Concurrency Control protocols, such as Locking (2PL) or Timestamp Ordering.
- **Levels:** SQL standards define isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable) which allow developers to trade strict isolation for performance.¹

2.1.4 Durability

Durability guarantees that once a transaction commits, its effects are permanent, even in the event of a subsequent system failure (crash, power loss).

- **Mechanism:** Before a transaction is confirmed as committed to the user, its log records must be flushed to non-volatile stable storage (disk). This "Force Log" policy ensures that even if the database pages in RAM are lost, the transaction can be "redone" from the log during recovery.²

2.2 Transaction States

A transaction models the lifecycle of a process. Understanding state transitions is crucial for implementing recovery managers.

State	Description	Transition Trigger
Active	The initial state where the transaction stays while it is executing read and write operations.	Begin Transaction
Partially Committed	The state reached after the final statement has been executed. Data changes may still reside in volatile memory.	End of last operation.
Failed	The state reached if normal execution can no longer proceed (e.g., hardware failure, logical error).	Error detection.
Aborted	The state after the transaction has been rolled back and the database restored to its prior state.	Rollback completion.
Committed	The state after successful completion where updates are permanently stored.	Successful writing of log/data to stable storage.

State Transition Logic:

1. A transaction starts in **Active**.
2. If all operations succeed, it moves to **Partially Committed**.
3. If the system can write necessary data/logs to disk, it moves to **Committed** (Success).
4. If an error occurs in *Active* or *Partially Committed*, it moves to **Failed**.
5. From *Failed*, the recovery manager rolls back changes, moving the transaction to **Aborted**.
6. An *Aborted* transaction can be restarted (if the error was transient, like a deadlock) or killed (if the error was logical, like bad input).¹

2.3 Serializability of Schedules

A **schedule** is a sequence of instructions that specifies the chronological order in which instructions of concurrent transactions are executed. When transactions are executed concurrently, their operations are interleaved. The goal of the DBMS is to create a schedule that allows parallelism (for efficiency) but is logically equivalent to a serial execution (for correctness).

2.3.1 Serial vs. Concurrent Schedules

- **Serial Schedule:** A schedule where the operations of each transaction are executed consecutively without any interleaving. For transactions \$T_1\$ and \$T_2\$, a serial schedule executes all of \$T_1\$ followed by all of \$T_2\$, or vice versa.
 - *Benefit:* Always consistent.
 - *Drawback:* Extremely poor resource utilization (CPU waits for I/O).

- **Concurrent Schedule:** Operations of T_1 and T_2 are interleaved.
 - *Benefit:* High throughput and utilization.
 - *Risk:* Can lead to anomalies like Lost Update, Dirty Read, or Inconsistent Retrieval.¹

2.3.2 Conflict Serializability

This is the most widely used criterion for correctness. A schedule is **Conflict Serializable** if it is conflict-equivalent to a serial schedule.

Conflict Definition:

Two operations O_i and O_j from transactions T_i and T_j conflict if:

1. They belong to different transactions ($i \neq j$).
2. They access the same data item (Q).
3. At least one of them is a WRITE operation.

Conflicting Pairs:

1. **Read-Write (RW):** T_i reads Q , then T_j writes Q . (Potential for Unrepeatable Read).
2. **Write-Read (WR):** T_i writes Q , then T_j reads Q . (Potential for Dirty Read).
3. **Write-Write (WW):** T_i writes Q , then T_j writes Q . (Potential for Lost Update).¹

Testing for Conflict Serializability (Precedence Graph):

To determine if a schedule SS is conflict serializable, we construct a directed graph $G = (V, E)$, called a precedence graph.

1. **Nodes:** Create a node for each transaction participating in schedule SS .
2. **Edges:** Draw a directed edge $T_i \rightarrow T_j$ if an operation in T_i conflicts with and precedes an operation in T_j .
 - If T_i executes Read(Q) before T_j executes Write(Q), draw $T_i \rightarrow T_j$.
 - If T_i executes Write(Q) before T_j executes Read(Q), draw $T_i \rightarrow T_j$.
 - If T_i executes Write(Q) before T_j executes Write(Q), draw $T_i \rightarrow T_j$.
3. **Cycle Detection:** If the graph contains a **cycle**, the schedule is **NOT** conflict serializable. If the graph is acyclic, the schedule is conflict serializable, and a topological sort of the graph gives the equivalent serial order.¹

2.3.3 View Serializability

View serializability is a less restrictive condition than conflict serializability. A schedule is View Serializable if it is **view equivalent** to a serial schedule. This concept primarily handles "Blind Writes" (writes without prior reads).

• **Conditions for View Equivalence:**

1. **Initial Read:** If T_i reads the initial value of Q in SS , it must also read the initial value in SS' .
2. **Read From:** If T_i reads a value of Q written by T_j in SS , it must read the value written by T_j in SS' .
3. **Final Write:** The transaction that performs the final write on Q in SS must also perform the final write in SS' .

- *Note:* Every conflict serializable schedule is view serializable, but not vice versa. However, testing for view serializability is NP-Complete, making it impractical for real-time scheduling.¹

2.4 Recoverability of Schedules

Beyond serializability, schedules must be recoverable. If a transaction fails, it should not leave the database in an inconsistent state or require the rollback of already-committed transactions (which is impossible).

2.4.1 Recoverable Schedules

A schedule is **Recoverable** if, for every pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i precedes the commit operation of T_j .

- **Logic:** T_j is dependent on T_i . If T_i aborts, T_j must also abort. If T_j commits before T_i , and then T_i aborts, T_j cannot be rolled back (since it committed), violating durability.
- **Rule:** $\text{Commit}(T_i) < \text{Commit}(T_j)$ if T_j reads from T_i .⁶

2.4.2 Cascadeless Schedules (ACA - Avoid Cascading Aborts)

A recoverable schedule allows **Cascading Rollbacks**. If T_1 writes A , and T_2 reads A , and T_3 reads A from T_2 , and then T_1 aborts, both T_2 and T_3 must also be aborted. This wastes significant work.

- **Definition:** A schedule is Cascadeless if, for every pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears **before the read operation** of T_j .
- **Logic:** Transactions are only allowed to read committed data. This prevents dirty reads entirely.⁶

2.4.3 Strict Schedules

Strict schedules simplify recovery by ensuring that the "undo" process simply restores the before-image of the data.

- **Definition:** A schedule is strict if a value written by a transaction T_i is neither read nor overwritten by any other transaction T_j until T_i either commits or aborts.
- **Hierarchy:** Strict $\subset$ Cascadeless $\subset$ Recoverable $\subset$ All Schedules.⁶

3. Concurrency Control Techniques

Concurrency control mechanisms coordinate simultaneous access to shared resources to preserve isolation and serializability.

3.1 Locking Techniques

Locking is a pessimistic concurrency control method. A transaction must acquire a "lock" on a data item before accessing it.

3.1.1 Lock Modes

1. **Shared (S) Mode:** Used for reading. If T_i holds an S-lock on Q , T_i can read but not write Q . Other transactions can also acquire S-locks on Q simultaneously.

2. **Exclusive (X) Mode:** Used for writing. If T_i holds an X-lock on Q , T_i can read and write Q . No other transaction can hold any lock (S or X) on Q .

Lock Compatibility Matrix:

Current Lock	Request S	Request X
Shared (S)	Compatible	Conflict
Exclusive (X)	Conflict	Conflict
None	Compatible	Compatible

3.1.2 The Two-Phase Locking (2PL) Protocol

Simply using locks does not guarantee serializability. The 2PL protocol dictates *when* locks can be acquired and released.

- **Phase 1: Growing Phase:** A transaction may obtain locks but may not release any lock.
- **Phase 2: Shrinking Phase:** A transaction may release locks but may not obtain any new locks.
- **Lock Point:** The moment the transaction acquires its final lock (end of growing phase). The ordering of lock points determines the equivalent serial schedule.
- **Theorem:** Any schedule generated by a valid 2PL protocol is conflict serializable.¹

Variants of 2PL:

1. **Basic 2PL:** As defined above. Allows potential cascading aborts.
2. **Strict 2PL:** All **Exclusive (X)** locks are held until the transaction commits. This guarantees Strict Schedules (no cascading aborts for write dependencies).
3. **Rigorous 2PL:** All locks (S and X) are held until the transaction commits. This is easier to implement and ensures the serialization order matches the commit order.¹

3.2 Deadlock Handling

In a locking environment, a **Deadlock** occurs when a set of transactions is blocked, each waiting for a lock held by another transaction in the set (e.g., T_1 holds A, wants B; T_2 holds B, wants A).

3.2.1 Deadlock Detection

The system allows deadlocks to occur but periodically checks for them.

- **Wait-For Graph:** The system maintains a directed graph where nodes are transactions. An edge $T_i \rightarrow T_j$ exists if T_i is waiting for a lock held by T_j .
- **Cycle:** If the graph contains a cycle, a deadlock exists.
- **Recovery:** The system selects a **victim** to rollback.
 - *Victim Selection Criteria:* Youngest transaction, fewest cycles computed, or least number of locks held.
 - *Starvation:* If the same transaction is always picked as the victim, it starves. To prevent this, the number of previous rollbacks is included in the cost function.¹

3.2.2 Deadlock Prevention

These protocols ensure the system never enters a deadlock state, typically by using transaction timestamps ($TS(T)$).

1. Wait-Die Scheme (Non-Preemptive):

- Based on transaction age.
- If older transaction T_i requests a lock held by younger T_j : T_i is allowed to **Wait**.
- If younger transaction T_j requests a lock held by older T_i : T_j **Dies** (aborts and restarts).

2. Wound-Wait Scheme (Preemptive):

- If older transaction T_i requests a lock held by younger T_j : T_i **Wounds** T_j (forces T_j to rollback).
- If younger transaction T_j requests a lock held by older T_i : T_j is allowed to **Wait**.
- *Insight:* Wound-Wait is often preferred because it preemptively kills younger transactions preventing them from blocking critical older processes.¹

3.3 Timestamp Ordering Protocol

This protocol does not use locks, eliminating deadlocks entirely. It orders transactions based on their start timestamps.

• Variables:

- $W\text{-}TS(Q)$: Largest timestamp of any transaction that successfully executed $Write(Q)$.
- $R\text{-}TS(Q)$: Largest timestamp of any transaction that successfully executed $Read(Q)$.

Protocol Rules:

1. Transaction T_i issues $Read(Q)$:

- If $TS(T_i) < W\text{-}TS(Q)$, it means Q has been overwritten by a younger transaction. T_i is too old. **Action: Reject and Rollback T_i .**
- Otherwise: Execute read and update $R\text{-}TS(Q) = \max(R\text{-}TS(Q), TS(T_i))$.

2. Transaction T_i issues $Write(Q)$:

- If $TS(T_i) < R\text{-}TS(Q)$, it means a younger transaction has already read the current value. Writing now would invalidate that read. **Action: Reject and Rollback T_i .**
- If $TS(T_i) < W\text{-}TS(Q)$, it means a younger transaction has already written a newer value. **Action: Reject and Rollback T_i** (Standard Rule).
- Otherwise: Execute write and update $W\text{-}TS(Q) = TS(T_i)$.¹

Thomas Write Rule:

This is an optimization for the $Write(Q)$ check. If $TS(T_i) < W\text{-}TS(Q)$, instead of rolling back, we can simply ignore the write.

- *Reasoning:* If a younger transaction wrote Q and no intervening transaction read the value T_i intended to write, then T_i 's write is effectively obsolete. Skipping it does not violate serializability and saves a rollback.¹

3.4 Granularity of Data Items and Intention Locks

Locking granularity refers to the size of the data item locked (Database, Table, Page, or Row).

- **Coarse Granularity (e.g., Database Lock):** Low concurrency, low locking overhead.
- **Fine Granularity (e.g., Row Lock):** High concurrency, high locking overhead.

Multiple Granularity Locking (MGL):

Allows transactions to lock at different levels. To manage this efficiently, MGL uses Intention Locks to signal updates at lower levels of the tree.

1. **Intention Shared (IS):** Explicit locking at a lower level with shared locks.
2. **Intention Exclusive (IX):** Explicit locking at a lower level with exclusive locks.
3. **Shared & Intention Exclusive (SIX):** The node is locked in Shared mode (implicitly locking all descendants in S), plus explicit locking is done at a lower level in X mode.¹²

Lock Compatibility Matrix:

Mode	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

Rules:

- Locks must be acquired root-to-leaf.
- To acquire an S or IS lock on a node, the parent must be locked in IX or IS.
- To acquire an X, IX, or SIX lock on a node, the parent must be locked in IX or SIX.
- Locks must be released leaf-to-root.¹²

4. Database Recovery Systems

Recovery systems ensure the **Atomicity** and **Durability** properties. They handle three main types of failures:

1. **Transaction Failure:** Logical errors (e.g., division by zero) or system enforcement (e.g., deadlock victim).
2. **System Crash:** Loss of volatile memory (RAM) due to power failure or OS crash.
3. **Disk Failure:** Catastrophic loss of non-volatile storage.²

4.1 Log-Based Recovery

The most common recovery technique relies on a **Write-Ahead Log (WAL)**. The log is a sequence of records stored on stable storage recording all updates.

- **WAL Rule:** Before a data item in the database (disk) is modified, the corresponding log record must be written to stable storage.

4.1.1 Deferred Update (No-Undo/Redo)

In this scheme, updates are recorded in the log but are **deferred** (not applied to the disk) until the transaction commits.

- **Execution:** When T_1 writes A , the log record $\langle T_1, A, 500 \rangle$ is written, but the disk block for A remains unchanged.
- **Commit:** Once T_1 commits, the log is flushed, and then the updates are applied to the disk.
- **Recovery:**
 - If the system crashes before commit, no changes were made to the disk. **No Undo needed.**
 - If the system crashes after commit but before updates were written, the system reads the log and **Redoes** the writes.¹⁵

4.1.2 Immediate Update (Undo/Redo)

In this scheme, updates can be applied to the database disk *before* the transaction commits, provided the log record is written first.

- **Execution:** Updates are written to disk as soon as feasible to free up buffer space.
- **Recovery:**
 - **Undo:** If the log contains $\langle T_i, \text{Start} \rangle$ but no $\langle T_i, \text{Commit} \rangle$, the transaction was active during the crash. The system uses the "Old Value" field in the log to revert changes (Undo).
 - **Redo:** If the log contains $\langle T_i, \text{Commit} \rangle$, the transaction must be durable. The system uses the "New Value" field to re-apply changes (Redo).⁷

4.2 Checkpoints

In a busy system, the log grows indefinitely. Replaying the entire log from the beginning after a crash is impossible. **Checkpoints** provide a known consistent state to shorten recovery time.

- **Checkpointing Steps:**
 1. Suspend execution of transactions (in static checkpoints).
 2. Flush all log records from memory to stable storage.
 3. Flush all modified buffer blocks (dirty pages) to the database disk.
 4. Write a $\langle \text{Checkpoint} \rangle$ record to the log.
 5. Resume transactions.
- **Recovery Optimization:** The recovery manager scans backwards to the last checkpoint. Only transactions active at or started after the checkpoint need to be processed (Redo or Undo). Transactions committed before the checkpoint are guaranteed to be on disk.²
- **Fuzzy Checkpoints:** Modern systems (like ARIES) use "fuzzy" checkpoints that do not halt the system. They record the state of the buffer pool (Dirty Page Table) and active transactions (Analysis Pass) to facilitate recovery without stopping service.¹⁸

4.3 Shadow Paging

An alternative to logging. The database maintains two page tables: **Current** and **Shadow**.

- **Operation:** When a transaction modifies a page, it writes to a *new* block and updates the Current page table. The Shadow table still points to the old block.
- **Commit:** The system atomically switches the root pointer from Shadow to Current.
- **Failure:** The system simply discards the Current table and points back to Shadow. No Undo/Redo is needed.
- **Disadvantage:** Leads to data fragmentation and poor locality of reference.²

4.4 Recovery from Catastrophic Failures

For media failures (disk crashes), log-based recovery is insufficient if the log itself is lost.

- **Archival Dumping:** Periodically dumping the entire database to remote storage (tape/cloud).
- **Incremental Backups:** Backing up only modified pages since the last dump.
- **Recovery:** Restore the last full dump $\rightarrow$ Apply incremental backups $\rightarrow$ Apply surviving transaction logs.¹⁹

5. Object-Oriented DBMS (OODBMS)

Traditional RDBMSs face an "impedance mismatch" when dealing with complex applications like CAD, GIS, or multimedia. Data is stored in flat tables, but applications use complex objects (inheritance, nesting). OODBMS bridges this gap.

5.1 Core Concepts

- **Object Identity (OID):** Every object has a unique, system-generated, immutable identifier. Unlike Primary Keys (which are value-based and can change), OIDs are independent of the object's state.²¹
- **Encapsulation:** Objects bundle data (attributes) and behavior (methods) together. Access is restricted to defined interfaces, enhancing security and modularity.³
- **Complex Data Types:** Support for non-atomic types like Sets, Lists, Bags, and Arrays. An object can contain other objects directly (nesting).²³
- **Inheritance:** Classes can be organized in a hierarchy (e.g., Student inherits from Person). This supports polymorphism and code reuse.²⁴

5.2 RDBMS vs. OODBMS Comparison

Feature	RDBMS	OODBMS
Data Structure	Normalized Relations (Tables)	Objects, Classes, Hierarchies
Relationships	Foreign Keys (Value-based)	Object References (Pointers)
Query Language	SQL (Declarative)	OQL (Object Query Language)

Feature	RDBMS	OODBMS
Access Speed	Slower for complex joins	Faster for graph traversals (pointer chasing)
Flexibility	Rigid Schema	Flexible, Extensible Schema

6. Distributed Data Base Management Systems (DDBMS)

A DDBMS manages a single logical database split across multiple physical locations connected by a network.

6.1 DDBMS Architecture and Transparencies

- **Architecture:** Can be **Homogeneous** (all sites use same DBMS, e.g., Oracle) or **Heterogeneous** (sites use different DBMSs, schema).
- **Transparencies:** The system should hide the distribution details from the user.
 1. **Location Transparency:** User queries data without knowing its physical site.
 2. **Fragmentation Transparency:** User sees one table Employee, unaware it is split into Emp_NY and Emp_LDN.
 3. **Replication Transparency:** User updates X, unaware that X is copied to 5 sites.⁴

6.2 Fragmentation and Replication

- **Horizontal Fragmentation:** Dividing a table by rows (Selection). E.g., $\sigma_{City='NY'}(Employee)$.
- **Vertical Fragmentation:** Dividing a table by columns (Projection). E.g., $\pi_{ID, Name}(Employee)$ and $\pi_{ID, Salary}(Employee)$. Must include Primary Key in all fragments for reconstruction.
- **Replication:** Storing data copies at multiple sites to improve availability.
 - **Synchronous:** All copies updated immediately (Slow, Consistent).
 - **Asynchronous:** One copy updated, others updated later (Fast, Eventual Consistency).²⁶

6.3 Distributed Transactions (Two-Phase Commit - 2PC)

To ensure atomicity across distributed sites, 2PC is used.

1. **Phase 1 (Prepare):** Coordinator sends "Prepare" to all sites. Sites execute transaction and lock resources, replying "Ready" or "Abort".
2. **Phase 2 (Commit):**
 - If *all* sites reply "Ready", Coordinator sends "Global Commit".
 - If *any* site replies "Abort" (or timeout), Coordinator sends "Global Abort".⁴

7. Overview of Database Security

Database security protects the confidentiality, integrity, and availability of data.

7.1 The CIA Triad

- **Confidentiality:** Preventing unauthorized disclosure. (Encryption, Access Control).
- **Integrity:** Preventing unauthorized modification. (Constraints, checksums).
- **Availability:** Ensuring data access. (Redundancy, DDoS protection).⁵

7.2 Access Control

1. **Discretionary Access Control (DAC):** The owner determines access (SQL GRANT/REVOKE). Flexible but vulnerable to Trojan horses.
2. **Mandatory Access Control (MAC):** System-wide policies based on security labels (Top Secret, Confidential). Rigid and secure.
3. **Role-Based Access Control (RBAC):** Privileges assigned to Roles (Manager, Clerk). Users assigned to Roles. Best for enterprise administration.²⁹

7.3 Threats and Defense

- **SQL Injection:** Injecting malicious SQL into queries. Defense: Prepared Statements.
- **Inference:** deducing sensitive info from aggregate statistical queries. Defense: Query restrictions, noise addition.
- **Data at Rest Encryption:** Encrypting DB files (TDE) to protect against media theft.³⁰